Policios, Andre C. **03/07/2026**

1. GitHub Repository Link

Your repository name must follow this format: IT342-Lastname-AppName

Backend Project Naming Convention (Spring Boot)

The backend project must follow the Maven naming convention below.
Group ID format: edu.cit.lastname
Examples:

- edu.cit.delacruz
- edu.cit.santos
- edu.cit.garcia

Artifact ID format: appname
Examples:

- campusclinic
- busticketing
- studenttracker

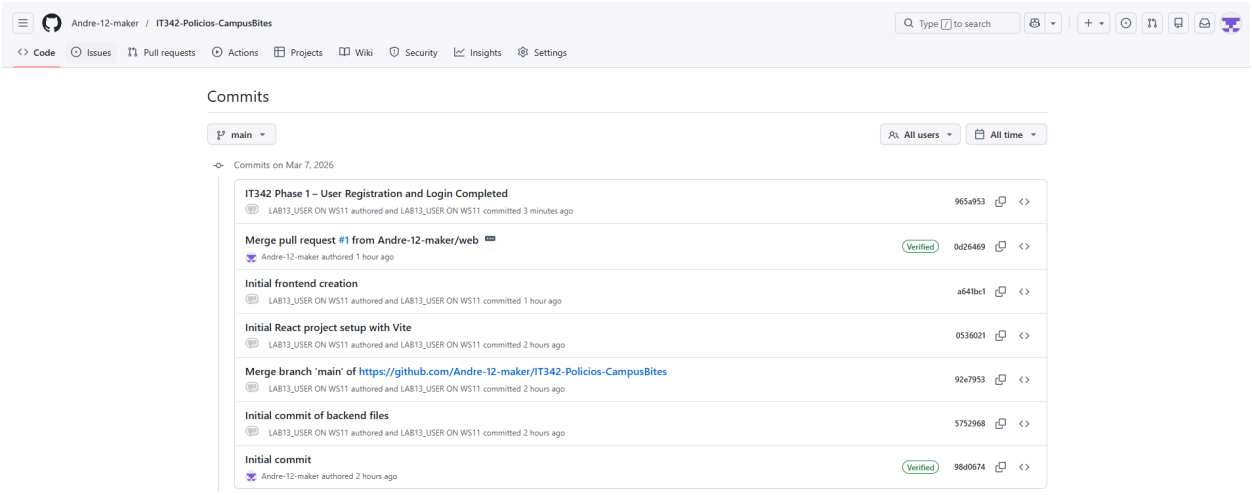
Example Maven Configuration
Group: edu.cit.delacruz
Artifact: campusclinic
This will generate a base package: `edu.cit.delacruz.campusclinic`
The backend must follow the required technology stack below.

- Framework: Spring Boot
- Version: Spring Boot 3.5.x
- Build Tool: Maven
- Architecture: REST API

2. Final Commit for Phase 1

Before the deadline, create a final commit in your repository.
Use the commit message:
IT342 Phase 1 – User Registration and Login Completed
Include the commit link or commit hash in your PDF.

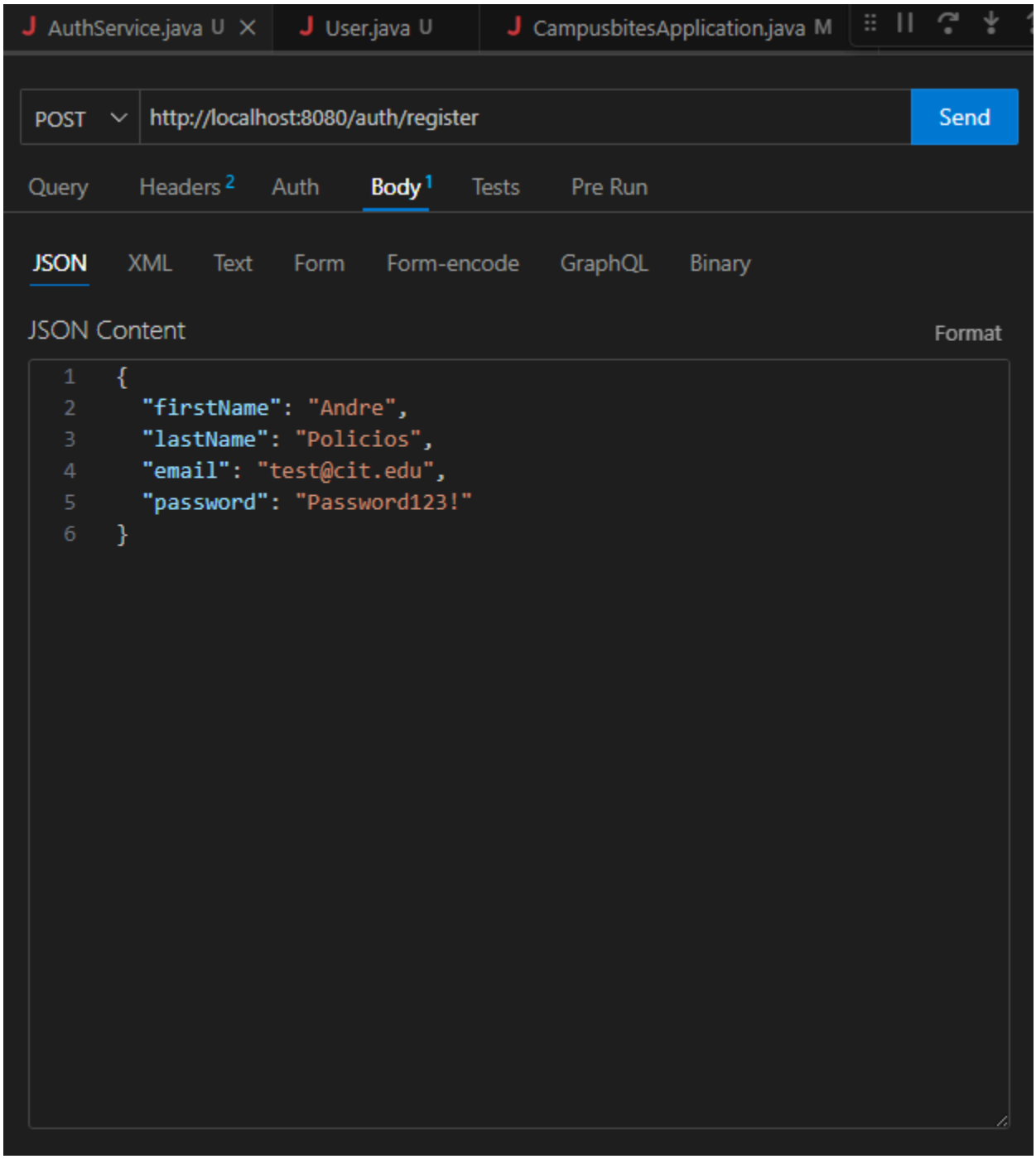
965a95312a1bc84821813b348f61cf3786cd0f91



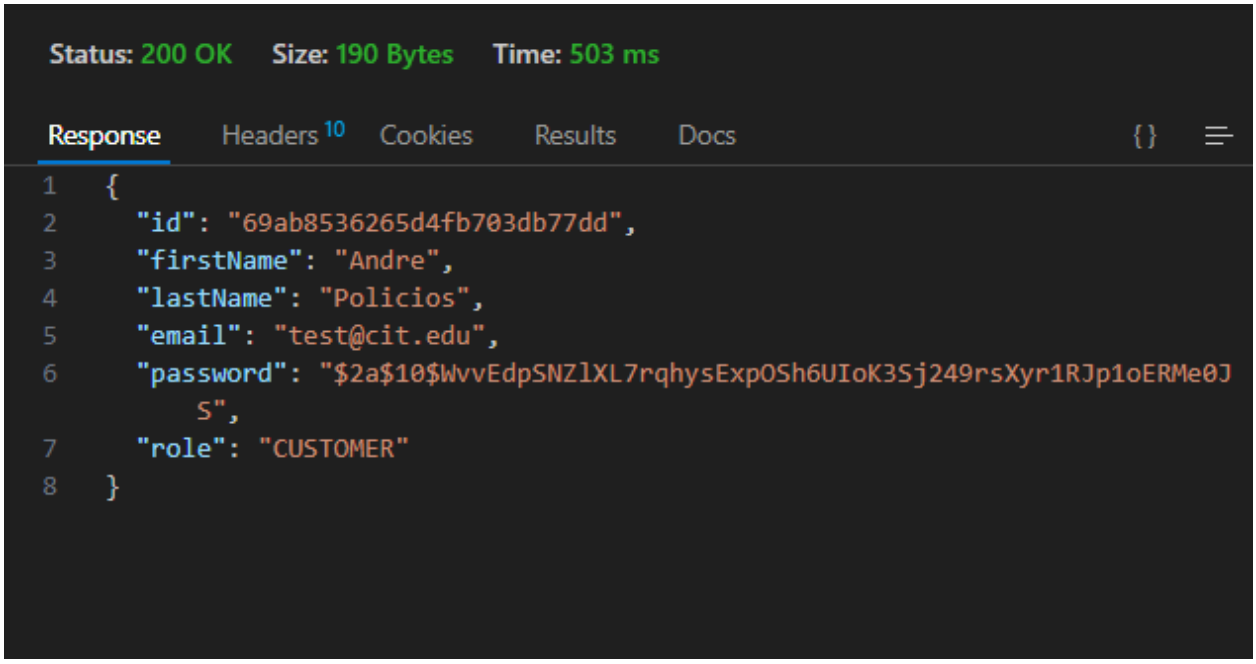
3. Screenshots of Implementation

Include screenshots showing the system working.
Required screenshots:

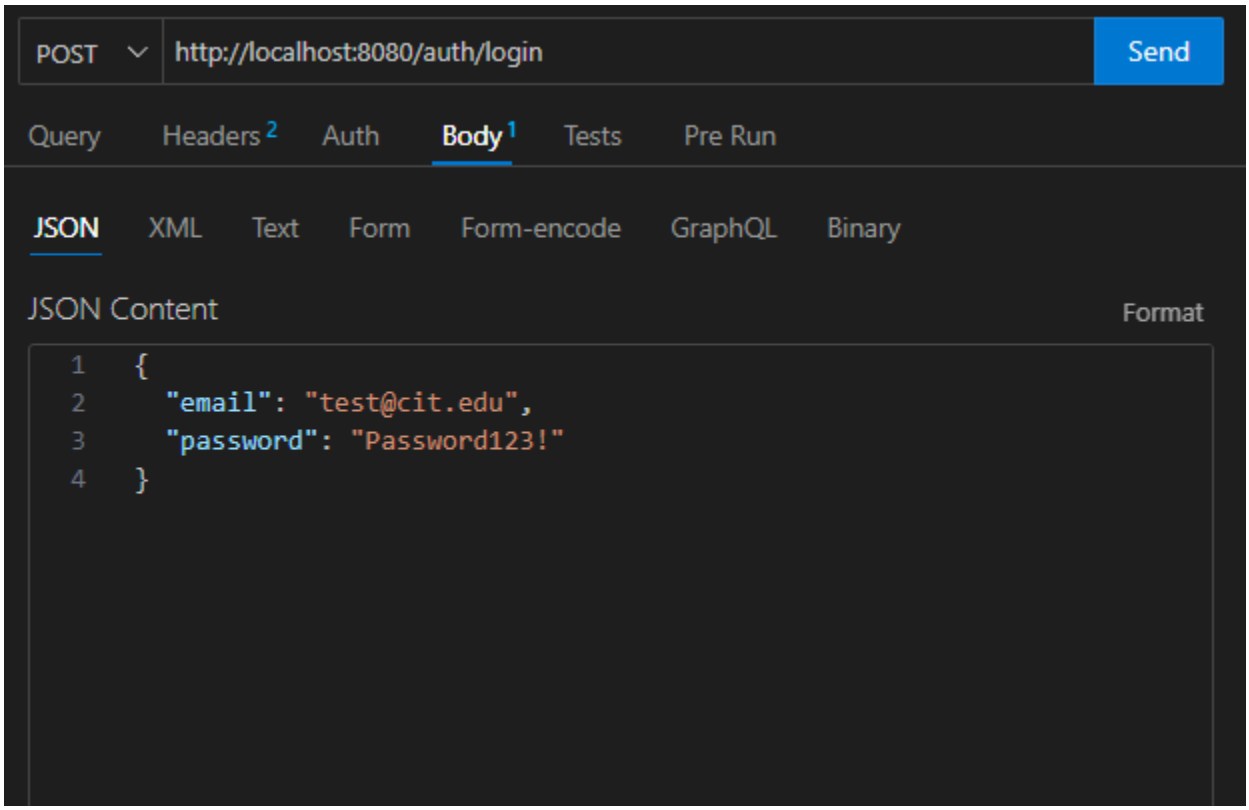
1. Registration page



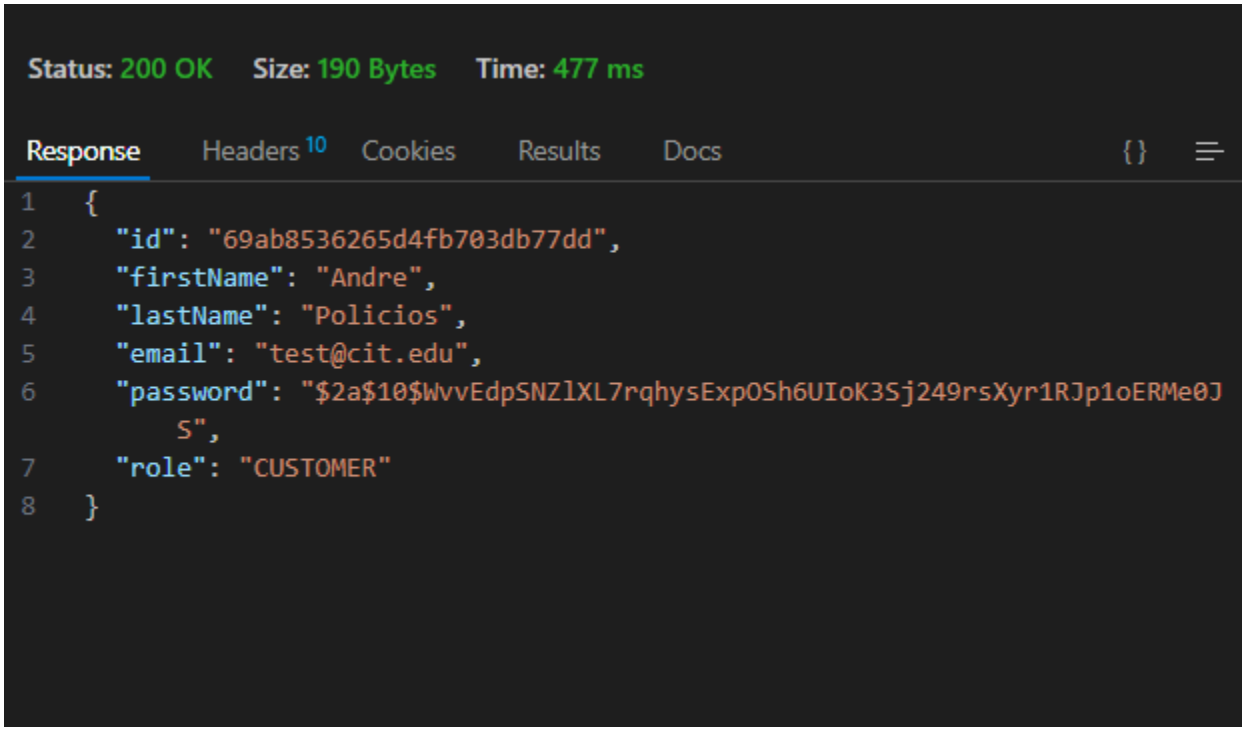
2. Successful user registration



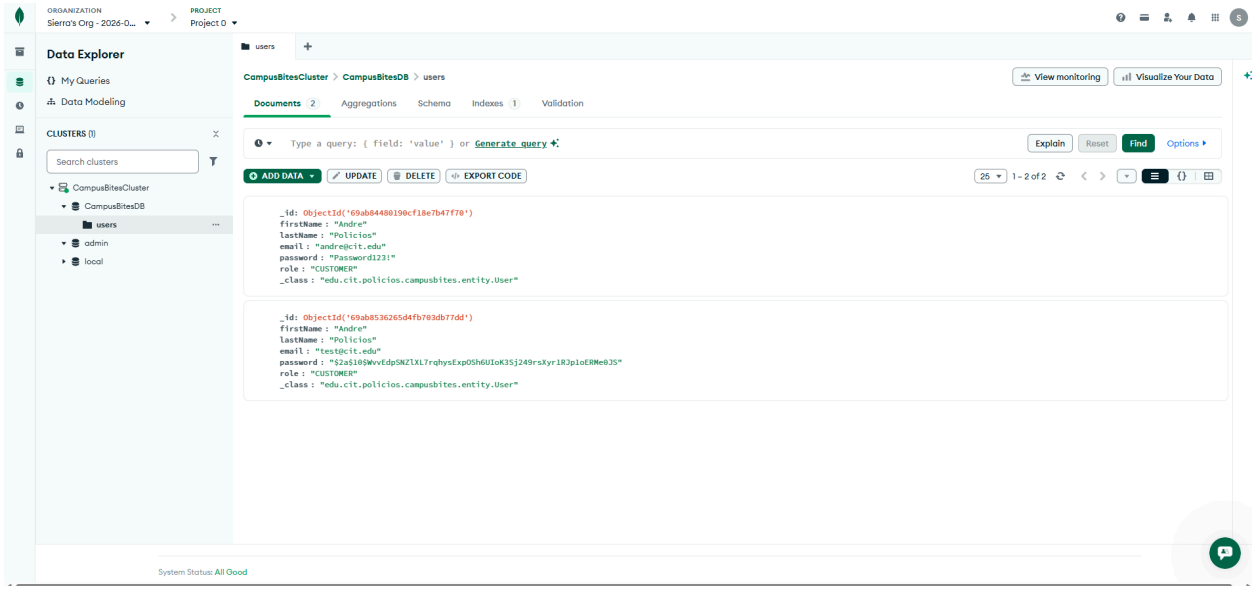
3. Login page



4. Successful login



5. Database record showing the registered user



4. Short Implementation Summary (1 Page)

User Registration

- **Registration Fields**: The system captures `firstName`, `lastName`, `email`, and `password` from the client via a JSON payload.
- **Validation Process**: The `AuthService` performs a check against the `UserRepository` to ensure the provided email does not already exist in the database.
- **Duplicate Prevention**: If an email is found, the system throws a runtime exception, preventing the creation of duplicate accounts.
- **Security**: Passwords are never stored in plain text. They are encrypted using the **BCrypt hashing algorithm** before being saved to MongoDB, fulfilling the security requirements of the SDD.

User Login

- **Login Credentials**: Users authenticate by providing their registered `email` and `password`.
- **System Verification**: The backend retrieves the user record by email and utilizes `passwordEncoder.matches()` to verify the raw input against the stored BCrypt hash.
- **Successful Login**: Upon successful verification, the server returns a **200 OK** status along with the user's details and their assigned role.

Database Table

- **Table/Collection**: `users` (Hosted on MongoDB Atlas).
- **Columns/Fields**: `id`, `firstName`, `lastName`, `email`, `password` (hashed), and `role`.

API Endpoints

- **POST** `/auth/register`: Handles new user creation and password hashing.
- **POST** `/auth/login`: Handles user authentication and credential verification.